Chapter 18 exercises • The Linux system

OS18-E01 • Fork and exec

Fix a single-threaded Linux 3.2 parent with one resident, writable private anonymous page. After fork, parent and child have separate mm mappings; the child only reads that page before immediate exec and no other writer changes it. Exclude MAP_SHARED, special mappings, and non-inherited VMAs.

1. How does COW reduce cost?
2. Which task receives the new image?
3. How must failed exec be handled?

OS18-E02 • Module references

In this teaching driver, file_operations.owner retains the module for one active handle, and one queued request/callback holds another reference under the driver protocol. Assume no other references and an ordinary, non-forced unload.

1. Can it unload now?
2. Which events must finish first?
3. Can signatures prevent use-after-free?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

OS18-E03 • VFS and page cache

A regular-file read returns 4 KiB from page cache. Compare two paths before power loss: (a) no sync, and (b) a successful fsync of that file. Assume the filesystem and device honor flush/durability contracts; other files are outside the scope.

1. What visibility does read success prove?
2. Which states may remain undurable?
3. What does fsync change?

OS18-E04 • Credentials and regular-file DAC

For Linux v3.2 generic_permission, fix a non-immutable regular file owned by 1000:100, mode 0400. The caller current_user_ns equals inode_userns and its relevant capabilities apply there. The mount is rw, no ACL/LSM denies, path traversal succeeded, and each independent request tests a single read, write, or execute DAC bit.

1. Compare read/write/execute for fsuid 1000 without capabilities, fsuid 0 with DAC_OVERRIDE, fsuid 0 without DAC capabilities, and fsuid 2000 with DAC_READ_SEARCH only.
2. Why does mode 0401 change capable execute checking?
3. Does seteuid(1000) permanently drop authority from real 1000/effective 0/saved 0?

OS18-E05 • Timeout, EIO, and late completion

In a single-terminal-notification model, R11 generation 3 is in flight with DMA to F40 for 4096 B. No successful bytes were previously delivered or credited to the upper layer; the EIO contract reports transferred=0. DMA may nevertheless have written part of F40. One consumer reference remains.

1. Can timeout immediately unmap/free? Give a counterexample.
2. After hardware stops and completes EIO, what are notification count and transferred bytes?
3. Handle duplicate same-generation completion and old generation 3 against new generation 4.
4. Can F40 be reused after terminal callback while consumer still holds it?

OS18-E06 • Clone sharing and ELF zero-fill

Initially one parent task has TID/TGID 100, its own mm and fd table, fd4→O1, and no other sharing references. A valid aligned ELF PT_LOAD does not overlap other segments: offset 0, vaddr 0x400000, filesz 0x1800, memsz 0x3000, page 4 KiB.

1. Count tasks, mm, fd tables, and original descriptions for fork versus VM/SIGHAND/THREAD/FILES clone.
2. How does v3.2 treat THREAD without SIGHAND, and SIGHAND without VM?
3. Calculate file/zero bytes per page. Does VMA creation imply residency?
4. Contrast exec failure before flush_old_exec and after its irreversible point.

OS18-E07 • IRQ reentry and unload races

CPU0 process holds L; its local IRQ handler and CPU1 also obey L. In a specified blocking, non-forced unload model, M holds one reference per handle/request, initially 3 handles and 2 requests with no other references. GOING rejects new references but allows existing work to drain.

1. Why can preemption disable alone still deadlock locally?
2. Does local IRQ disable exclude CPU1? Combine appropriate mechanisms.
3. Trace LIVE→GOING→close handles→finish requests→callback quiescence, with counts/free conditions.
4. Why is checking zero before withdrawing entry unsafe?

OS18-E08 • TCP segmentation and latency causes

In one IPv4/TCP trace, MTU=1500, IP/TCP headers are 20 B each, effective MSS=1460 B, and there is no offload, retransmission, or extra segmentation. A 6000 B send is packetized as four 1460 B segments followed by 160 B. One segment has enqueue, dispatch, transmit-complete, and ACK times 0, 2, 3, and 13 ms.

1. Give MSS, payloads, and L3 bytes excluding Ethernet/ACK/retransmissions.
2. Does acceptance prove remote processing?
3. Split queue, device, and subsequent ACK delay.
4. Another observation has queue length 20, retransmissions 3, device errors 1. Does it prove queue-caused tails? What more is needed?

OS18-E09 • Pipe EOF, signals, and semop

After fork both processes retain pipe writers; the child writes ABC and closes its writer. The consumer owns a valid read end and requests a positive count. One blocked standard signal is generated three times for the same pending queue; semaphores start S0=1 and S1=0.

1. Must EOF follow draining ABC? When does read return 0?
2. Must the handler run three times? Contrast realtime signals.
3. In atomic semop(S0-=1, S1-=1), does S0 first become 0 while S1 is insufficient?
4. Explain lost wakeup from checking before enqueue and the required correction.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz et al., Operating System Concepts, 9th ed., Ch. 18. Linux v3.2 fork, memory, module, DAC, ELF. fork(2), signal(7).

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study